1. Core Concept

Both External Volumes and External Tables work with data stored outside Databricks-managed storage.

But they solve different problems:

External Volume
→ Governed access to FILES

External Table
→ Governed access to STRUCTURED TABLE DATA

The practical flow in this class was:

External Location
↓
External Volume
↓
Read orders.csv
↓
Write Delta Files
↓
External Delta Table

---

2. What Is an External Volume?

An **External Volume** is a Unity Catalog object that provides governed access to files already stored in external cloud storage.

Example external storage:

abfss://datax@<storage-account>.dfs.core.windows.net/landing/orders/

After creating an External Volume, the same files can be accessed through:

/Volumes/datax_catalog/bronze/orders_external/

Important:

External Volume
≠ Copying files

The files remain in ADLS Gen2.

The volume simply provides a governed way to access them.

---

3. What Is an External Table?

An **External Table** is a registered table whose actual data files are stored at an external cloud location.

Example:

datax_catalog.bronze.orders_external_delta

The table can be queried using:

```
SELECT *
FROM datax_catalog.bronze.orders_external_delta;
```

Unity Catalog manages:

- Table metadata
- Permissions
- Governance

But the underlying files remain at the externally managed storage path.

---

4. External Volume vs External Table

External Volume	External Table
Used for files	Used for structured data
`/Volumes/catalog/schema/volume/`	`catalog.schema.table`
CSV, JSON, Parquet, images, etc.	Structured table data
File APIs / Spark reads	SQL / DataFrame queries
Good for ingestion/file exchange	Good for analytics and table operations

Easy Interview Answer

Incoming CSV/JSON files
→ External Volume

Processed Delta dataset
→ External Table

---

5. Creating an External Volume

Example:

```
CREATE EXTERNAL VOLUME IF NOT EXISTS
datax_catalog.bronze.orders_external
LOCATION
'abfss://datax@<storage-account>.dfs.core.windows.net/landing/orders/';
```

The important keyword is:

LOCATION

It points the Unity Catalog Volume to an existing external directory.

The path must be covered by an authorized External Location.

---

6. Reading Files Through the Volume

After the volume is created:

```
volume_path = (
    "/Volumes/datax_catalog/bronze/"
    "orders_external/"
)

display(dbutils.fs.ls(volume_path))
```

Read the file:

```
orders_path = (
    "/Volumes/datax_catalog/bronze/"
    "orders_external/orders.csv"
)
```

```
orders_df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(orders_path)
)
```

The file was not uploaded again.

The External Volume is pointing to the existing ADLS files.

---

7. Keep Raw Files and Delta Files Separate

A very important storage-design rule:

```
landing/orders/
→ Incoming CSV files

delta/orders_external/
→ Delta table files
```

Example:

```
datax/
├── landing/
│   └── orders/
│       └── orders.csv
│
└── delta/
    └── orders_external/
        ├── _delta_log/
        └── part-*.parquet
```

Do not mix source CSV files and Delta table files inside the same directory.

This makes storage management cleaner and avoids confusion.

---

8. Creating an External Delta Table

Example:

```
CREATE TABLE IF NOT EXISTS
datax_catalog.bronze.orders_external_delta
(
   order_id INT,
   customer_id STRING,
   product_id STRING,
   order_date DATE,
   quantity INT,
   unit_price DECIMAL(10,2),
   order_status STRING
)
USING DELTA
LOCATION
'abfss://datax@<storage-account>.dfs.core.windows.net/delta/orders_external/';
```

Key point:

Table Metadata
→ Unity Catalog

Actual Delta Files
→ External ADLS Location

---

9. Verify Whether a Table Is External

Useful commands:

```
DESCRIBE EXTENDED
datax_catalog.bronze.orders_external_delta;
```

and:

```
DESCRIBE DETAIL
datax_catalog.bronze.orders_external_delta;
```

Check:

- Table type
- Storage location
- Delta details

These commands are useful when joining an existing project and trying to understand how a table is configured.

---

10. What Happens When You DROP an External Object?

External Table

DROP TABLE
↓
Unity Catalog registration removed
↓
External files remain

External Volume

DROP VOLUME
↓
Unity Catalog registration removed
↓
External files remain

For external objects, Unity Catalog does not own the external data lifecycle in the same way as managed objects.

Important Interview Point

Dropping the external object does **not** mean the files can never be deleted.

Someone with sufficient storage permissions can still delete them directly.

---

11. Managed vs External Objects

Managed

Databricks / Unity Catalog
→ Manages storage lifecycle

External

Unity Catalog
→ Manages metadata + governance

External Storage
→ Lifecycle remains separately controlled

This lifecycle distinction is one of the most important things to remember.

---

12. When Should You Use an External Table?

External tables make sense when:

- Delta data already exists in ADLS Gen2
- Another team owns the storage
- Another platform manages the data lifecycle
- Existing storage paths must be preserved
- Migration requires keeping data in place

Example:

Existing Delta Dataset in ADLS
↓
Register as External Table
↓
Query through Databricks

No need to copy the entire dataset just to make it queryable.

---

13. When Is a Managed Table Better?

Managed tables are often a better default when:

- Databricks owns the full data lifecycle
- There is no requirement to control the physical storage path
- You are building a Databricks-first workload

The class takeaway is:

> Use external tables when you have a clear storage ownership or integration requirement.

---

14. Can MERGE Work on an External Delta Table?

Yes, provided:

- It is a Delta table
- The operation is supported
- The executing identity has the required permissions

So:

External
≠ Read-only

An External Delta Table can still support Delta operations such as:

MERGE
UPDATE
DELETE

subject to permissions and table capabilities.

---

15. Important Permissions

Creating External Volume

Requires appropriate permissions such as:

CREATE EXTERNAL VOLUME

on an authorized External Location.

Creating External Table

Typically requires:

CREATE TABLE

on the target schema and:

CREATE EXTERNAL TABLE

on the External Location.

Reading External Volume

Generally requires:

USE CATALOG

USE SCHEMA
READ VOLUME

subject to applicable access controls.

---

16. Common Troubleshooting Scenarios

External Volume Creation Fails

Check:

- `CREATE EXTERNAL VOLUME` privilege
- External Location coverage
- Storage URL
- Path overlap

External Table Creation Fails

Check:

- `CREATE EXTERNAL TABLE`
- `CREATE TABLE`
- Storage path access
- Existing incompatible files

Cannot Read External Volume

Check:

- `READ VOLUME`
- Azure storage access
- Networking
- Correct file path

Duplicate Records After Re-run

Reason:

append
→ inserts again

Solution:

- Deduplication
- Idempotent processing
- MERGE
- Processed-file tracking

---

17. Common Mistakes

1. Treating Volume as Table

Volume → Files
Table → Structured Data

2. Mixing CSV and Delta Files

Keep landing and Delta storage separate.

3. Using Same Path for Multiple Governed Objects

Avoid unsupported path overlaps.

4. Assuming DROP Deletes External Files

It normally removes the Unity Catalog registration, not the underlying external data.

5. Using External Tables for Everything

Managed tables are often better when Databricks owns the full lifecycle.

6. Re-running Append Without Thinking About Duplicates

Append does not automatically deduplicate data.

---

18. Interview Questions

Q1. What is an External Volume?

A Unity Catalog object that provides governed access to files stored in an existing external cloud-storage directory.

Q2. What is an External Table?

A registered table whose underlying data files are stored at an external cloud location.

Q3. What is the main difference between an External Volume and External Table?

External Volume
→ File access

External Table
→ Structured queries and table operations

Q4. Does creating an External Volume copy the files?

No.

It points to and governs access to the existing storage directory.

Q5. What happens when an External Table is dropped?

Its Unity Catalog registration is removed, while the underlying external files remain.

Q6. Why keep raw files and Delta files in separate directories?

Because they serve different purposes.

Delta directories also contain transaction metadata such as:

_delta_log

Separating them improves reliability and storage management.

Q7. Can we use MERGE on an External Delta Table?

Yes, if the operation is supported and the identity has the required privileges.

Q8. When would you prefer a managed table?

When Databricks owns the data lifecycle and there is no requirement to manage the physical storage path externally.

---

19. Scenario-Based Interview Questions

Scenario 1

A company already has a large Delta dataset in ADLS Gen2. They want Databricks to query it without copying it. What would you do?

Create or use an authorized External Location and register the existing Delta path as an External Table.

Scenario 2

Daily CSV files arrive in ADLS. External Volume or External Table?

Use an External Volume for governed access to the incoming files.

After processing the records into Delta, expose them through a Delta table.

Scenario 3

You drop an External Table by mistake. Is the data automatically gone?

No.

The table registration is removed, but the external files remain unless something separately deletes them.

Scenario 4

You rerun an append-based load and record count doubles. Why?

Because append inserts the records again.

The pipeline needs idempotent logic, deduplication, or MERGE depending on the requirement.

Quick Revision

External Location
→ Authorizes / governs cloud path

External Volume
→ Governed file access

External Table
→ Governed structured data

/Volumes/...
→ File access

catalog.schema.table
→ Table access

DROP External Object
→ Registration removed
→ External files remain

Managed
→ Databricks manages lifecycle

External
→ Storage lifecycle remains externally controlled

Final Takeaway

Use External Volumes when you need governed access to files, and External Tables when you need structured SQL/DataFrame access to data that should remain in externally managed cloud storage.